# CURATE: Leveraging LLM Agents to Compose, Catalog, and Deploy Reproducible Workflows

Nolan Cutler
*School of EECS*
*Oregon State University*
Corvallis, Oregon, USA
cutlern@oregonstate.edu

Chia-Chen Kuo
*School of EECS*
*Oregon State University*
Corvallis, Oregon, USA
kuochi@oregonstate.edu

Nanda Velugoti
*School of EECS*
*Oregon State University*
Corvallis, Oregon, USA
velugotn@oregonstate.edu

Kathryn Newhart
*School of CBEE*
*Oregon State University*
Corvallis, Oregon, USA
newhartk@oregonstate.edu

Renato Figueiredo
*School of EECS*
*Oregon State University*
Corvallis, Oregon, USA
figueren@oregonstate.edu

***Abstract*—Agentic code generation has the potential to accelerate the development of computational workflows while also reducing barriers to entry. However, a key gap remains: existing coding agents focus on code generation and do not address the entire workflow lifecycle, including deployment and sharing. As a result, users develop and stitch modules independently while managing deployment on their own. To address this gap, we propose CURATE (Composition, User-in-the-loop, Reuse, and Automated Task Execution), a novel human-in-the-loop multi-agent system that uses LLM agents to manage and develop composable workflows across their entire lifecycle. A key feature of the system is a catalog that allows for the storage and reuse of modules across workflows. Module catalogs provide a foundation that can be expanded to support FAIR principles by facilitating the sharing and reuse of curated modules and subgraphs. We demonstrate the feasibility of our system with an initial prototype and 6 experiments.**

***Index Terms*—scientific workflow, multi-agent, HITL, FAIR, coding agent**

## I. Introduction

Scientific workflows are a vital part of the scientific discovery process. They allow researchers to automate data ingestion and computational experiments, reproduce their results, and share their research artifacts with the broader scientific community [1], [2]. However, workflows require significant effort and time to build. Users must develop familiarity with workflow management tools, source or write workflow steps by hand, and manage deployment infrastructure. These systems have a steep learning curve that hinders adoption and takes time away from other research activities [3]. Opportunely, generative AI has shown promise in accelerating parts of the scientific software development process [4], [5]. Large language models (LLMs) can generate code from natural language, and systems such as LLM4Workflow [6] demonstrate their ability to compose multi-step workflows. Recently, LLM agents have been utilized for their ability to autonomously plan tasks and invoke tools [7]–[9], self-correct [10], and incorporate human expertise into their decision-making [11]. Moreover, coding agents, such as Claude Code and Codex, have demonstrated a remarkable ability to generate, test, and execute code locally on a user's device, with human-in-the-loop (HITL) verification [12], [13]. At the same time, initiatives such as the Workflow Community Initiative (WCI), WorkflowHub [14], and CWL [15] have pushed to imbue workflows with FAIR (Findable, Accessible, Interoperable, Reusable) principles [16]. Still, existing agent-driven workflow generation systems lack an end-to-end view of the scientific workflow lifecycle, from generating and composing workflow steps to deployment and monitoring to broader FAIR reuse and sharing of validated components. The contributions of our work are the design and proof-of-concept implementation of the core functionality of a novel multi-agent system for scientific workflow generation, incorporating existing components, as well as early experiments demonstrating its feasibility.

## II. Related Work

**Scientific Workflows**: Scientific workflows describe processes orchestrated to achieve a scientific goal. They are typically expressed as directed graphs of computational tasks with data dependencies and executed by a workflow management system (WMS) that handles scheduling, reproducibility, data movement, and fault tolerance [17]. A wide range of WMSs exists, from high-performance computing (HPC) targeted systems to cloud approaches [17]. Our proposed architecture uses an intermediate workflow representation – a skeleton – that is not inherently tied to any particular WMS.

**Agents for Workflow Generation**: Coding agents, such as Claude Code and Codex, are LLM-enabled agents for code generation [12], [13]. These systems integrate LLM inference, reasoning, and tool calling to autonomously iterate through code generation and execution, providing initial checks before human verification. Agents can use external tools and skills that can provide reusable instructions, scripts, and resources [7], [18]. Thareja et al. [19] apply skills to

SC26 Workshops, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-1221-5/26/$31.00 ©2026 IEEE

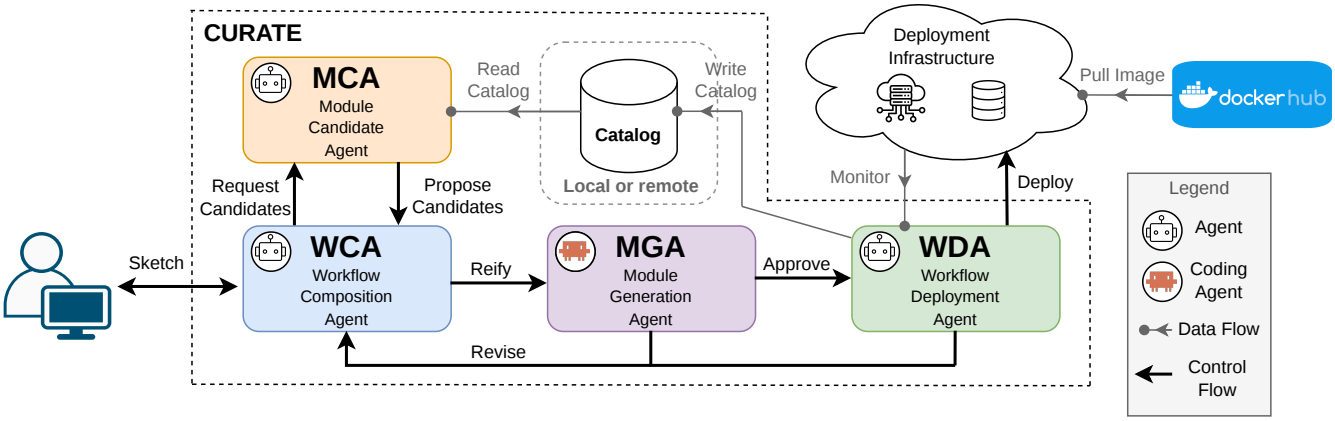



Fig. 1: Overview of the CURATE architecture. The user iterates with the multi-agent system through sketch → reify → test → deploy phases, with checkpoint gates allowing user verification and revision, and a catalog for code sharing and reuse.

Claude Code, Codex, and Kimi for workflow generation targeting Pegasus [20], demonstrating the potential for agentic workflow development. While sharing similarities, CURATE introduces a key novel capability: a catalog that supports storage, retrieval, and reuse of user-verified modules by the agentic system. Lee et al. [21] apply HITL and module reuse through Retrieval Augmented Generation; however, their system targets Excel and CSV spreadsheet data-cleaning tasks.

**FAIR and Workflow Cataloging**: Prior work has integrated FAIR principles into workflow development through workflow registries, platform-agnostic workflow languages, and workflow component search [14], [15], [22]. WorkflowHub [14] provides a registry of computational workflows for sharing and reuse. CWL [15] provides a partially platform-agnostic solution to workflow definition. Starlinger et al. [22] demonstrate the feasibility of similarity search for workflows, but do not provide a way to reuse components during workflow composition automatically. These projects are complementary to CURATE. WorkflowHub, for example, could serve as a foundation for creating pre-seeded catalogs of expert-vetted, auditable workflow components; CWL could be used as a target workflow representation to enable a wider range of WMS deployment options; and advanced catalog searching techniques could improve the accuracy of module sourcing.

**Commercial Science Assistants**: Google AI Co-Scientist [23] and Claude Science [24] are recent agentic systems that assist scientists with literature synthesis, data analysis and visualization, and manuscript drafting. In Claude Science, data analysis and visualization code runs on the user's infrastructure (including locally, over SSH, or via job schedulers), and provenance records are generated for the code, environment, and dependencies of all outputs. Unlike CURATE, these systems do not allow the generation of multi-step workflows targeting existing WMS, do not provide a way to reuse or share validated workflow components, and are locked-in to specific vendors.

**Connecting Agents to Cloud and HPC**: A growing body of work connects agents to cloud and HPC resources. Ma et al. [25] modified LangChain to allow agents to call functions scheduled on HPC resources using Parsl [26]. In Yildiz and Peterka [27], LLMs are evaluated on their ability to annotate, configure, and translate WMS files, demonstrating the potential for workflow generation and management. Looking beyond individual systems, Shin et al. [28] provide a broader overview of emerging workflow systems, arguing that workflows will evolve along two dimensions: composition (static to swarm) and intelligence (static to intelligent). Rather than connecting agents directly to distributed computing resources, CURATE uses agents to generate workflow executables that are reproducible and can be deployed using legacy WMS.

## III. Architecture

### A. Workflow Model

Workflows are composed of nodes and edges, where each node represents a computational task to be completed, and edges define control dependencies. The underlying WMS determines the type of these nodes. We refer to the implementation of these nodes as *modules*, which may be functions, scripts, or other types of executables with inputs and outputs. For our proposed architecture, task precedence must form a directed acyclic graph (DAG) and be known ahead of runtime scheduling. In Section IV, we present an implementation that, per prior workflow terminology [17], targets a WMS with function-granularity, task-driven, file-transported workflows, where nodes/modules are implemented as Python functions.

### B. Overview

Our proposed architecture (Fig 1) consists of 4 agents: a Workflow Composition Agent (WCA), a Module Candidate Agent (MCA), a Module Generation Agent (MGA), and a Workflow Deployment Agent (WDA). In addition, the architecture includes a catalog for reusing workflow modules. Catalog reuse is enabled through *source resolution*, which, in the context of CURATE, refers to the process by which the MCA determines a node's module source based on its natural-language description in the skeleton. The resolved module source can be one of the following: 1) the catalog (if a match is found for reuse/adaptation); 2) a user-provided

module; 3) a previous iteration of the workflow; or 4) a module to be generated by the MGA (if a match is not found). While the catalog itself can be extended as agentic memory [29], the current prototype implements it as passive storage. Workflow generation begins with the user providing a high-level workflow description to the WCA – a *sketch* – in natural language, and the system follows a sketch → reify → test → deploy set of stages, with the possibility of iterative HITL refining.

### C. Agent Descriptions

**Workflow Composition Agent (WCA)**: The WCA controls the structure of the workflow. Based on the user's request, it generates a high-level skeleton of the workflow with natural-language descriptions of each node, providing an overview of the precedence of each step. This intermediate representation is presented to the user for verification via a user interface, such as an interactive graphical user interface (GUI) or terminal user interface (TUI), and the user may provide feedback. This check enables fine-grained control over task decomposition and allows users to catch errors early, before the code generation process begins. After the skeleton is created, it delegates source resolution to the MCA. Finally, the WCA resolves skeleton precedence edges with concrete inputs and outputs.

**Module Candidate Agent (MCA)**: The MCA resolves sources for skeleton tasks. When the WCA requests candidates from the MCA, it provides a high-level skeleton of the workflow. The MCA queries the catalog to see what existing modules are available. For each action in the skeleton, it maps it to a source. The following resolutions are available: freshly generated, from the catalog, a previous workflow revision, or a user-provided module. To make these classifications, the MCA is provided with additional workflow context, including the user's initial request, a description of the workflow management system, and the user's previous revisions. This agent enables users to reuse previously verified modules, either from their own previous runs or from other trusted users.

**Module Generation Agent (MGA)**: The MGA is tasked with implementing workflow modules. It uses a coding agent to generate or adapt modules and to test them locally in a scratch directory, skipping nodes that are reused from the catalog or previous revisions. The MGA must ensure that each implemented module's I/O matches the WCA skeleton.

**Workflow Deployment Agent (WDA)**: The WDA deploys and continually monitors the workflow and its uploaded artifacts. After the workflow execution completes, the WDA spawns a post-run chatbot agent. This agent provides the user with natural language context about the run, including any failures that occur. The user can chat with this agent to ask questions about the run, and the agent invokes Model Context Protocol (MCP) [30] tools to download artifacts and view logs before providing a natural-language response. This agent can also provide revision suggestions, including pointing out incorrect configurations, missing files, or genuine code issues, and ask the user for approval to revise.

### D. Catalog

Cataloging serves as the basis for reusing workflow modules. When a user finishes composing a workflow, they are prompted to store it in the catalog. Depending on the target WMS, catalogs may consist of functions, ad hoc scripts, or, in future work with emerging workflow systems, agents with access to scientific instruments [28], [31]. During source resolution, the MCA queries the catalog and reuses entries where appropriate. Catalog entries may be stored locally on the user's file system or on cloud or remote storage. Users can share these catalogs by copying entries from other trusted users, serving as a foundation for future work that supports workflow reusability, discovery, and interoperability.

### E. User Supplied Module Reuse

When the user chooses to provide an existing module, the MCA will determine whether to use it verbatim or adapt it. This selection is based on whether the module itself needs to change and excludes incompatible formatting for the underlying WMS. If the MCA determines to include a user-provided module verbatim, the MGA wraps it with I/O and the appropriate annotations, but does not change the module.

### F. Resource Isolation

To mitigate the risk of agent-generated actions causing harm to user resources or data, or interfering with other processes, our proposed architecture ideally targets runtimes in which individual tasks can be sandboxed to isolated environments, such as containers or virtual machines. While these environments provides a foundation for the isolated execution of CURATE modules, care must also be taken to enforce isolation of state committed to centralized storage to prevent the propagation of side effects through file inputs/outputs that break downstream data dependencies. Future work will seek to incorporate new and existing abstractions for resource isolation into the AI-assisted workflow development process, from workflow definition to runtime environment.

## IV. Prototype

We implemented an initial open-source [32] prototype in 6700 lines of Python, using LangGraph for agent coordination and Claude Opus 4.8 as our underlying LLM. This prototype represents a minimal implementation of our proposed architecture, and as such, remains feasibility-focused. The MGA uses the Claude Agent SDK to generate function implementations; however, the prototype is extensible to support open-source coding agents, such as Codex, and open-source LLMs, with minimal changes. For deployment, we chose to target FaaSr [33], a serverless FaaS WMS (originally targeting R, but since extended to Python), for the following reasons. First, FaaSr uses a schema-defined, task-driven, function-granularity DAG workflow model, with the only dynamicity being conditional branches, meaning all workflow structure and tasks are known ahead of time. The result is a WCA skeleton that can be represented as a simple DAG of functions. Second, serverless simplifies deployment, allowing our WDA to invoke

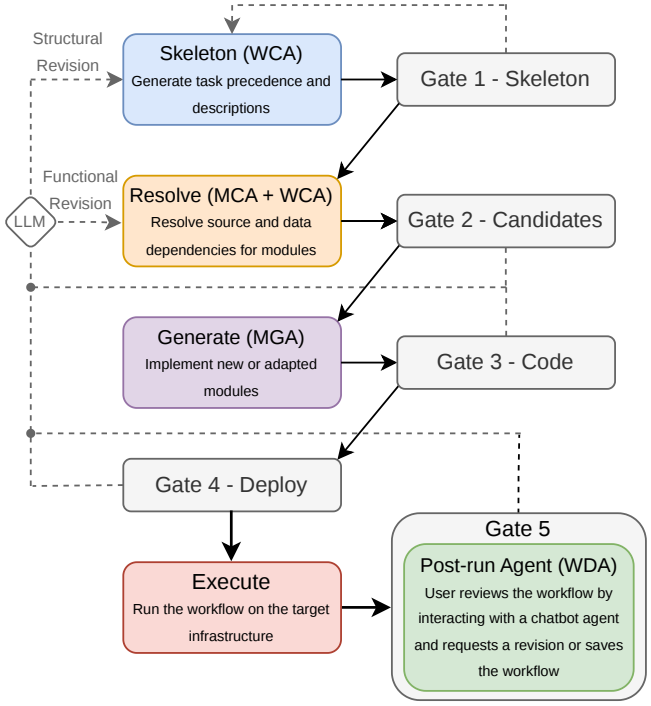



Fig. 2: A visualization of the 9-stage LangGraph pipeline, with 5 HITL gates for workflow revision. An LLM is used to determine whether revisions require structural changes or only affect functions already present in the skeleton.

the workflow without complex resource management. Finally, serverless deployment provides isolation via containerized execution of modules. The prototype exposes a command-line interface (CLI) app with a TUI viewer for code review and workflow monitoring. Since in serverless FaaS a module is expressed as a function, we use the terms *module* and *function* interchangeably in the following discussion.

### A. Pipeline and HITL Gates

The LangGraph pipeline comprises 9 stages and 5 HITL gates, where users can provide feedback for revision, as shown in Figure 2. At each gate, the user must approve the system's output and may provide optional revisions before moving on.

**Skeleton and Gate 1**: The WCA generates a candidate skeleton that outlines the workflow's structure and high-level steps in natural language as sketched by the user. At this stage, the user can provide their own modules for reuse and context such as PDF documents, example I/O, or markdown files.

**Resolution and Gate 2**: After the user approves the skeleton at gate 1, the WCA fills in descriptions for the input and output edges between functions. Then, control shifts to the MCA, which determines an appropriate source for each function in the skeleton. First, it retrieves a list of potential functions from the catalog. Then it uses an LLM call to determine whether each node in the workflow should be resolved to the catalog, a previous iteration, or generated. If it chooses an existing function, it also determines whether it needs to be adapted to meet the function's requirements or can be reused verbatim. Then it returns its selections to the WCA, and the WCA fills in concrete dataflow (file names) between nodes and provides a list of likely needed PyPI dependencies. The user is then prompted to approve the filled-in (but not fully reified) workflow at gate 2.

**Generation and Gate 3**: Next, the MGA performs a topological sort of the skeleton and traverses it sequentially to generate each workflow function in its own scratch directory. The directory contains a `CONTEXT.md` file detailing shared context about the workflow, the user prompt, data integrity requirements, and the local FaaSr testing stubs, along with a per-function file specifying each function's description, signature, input/output files and dataflow, dependencies, and secret names (e.g., API key names), if any. Due to the topological generation ordering, the MGA sees any upstream-generated or cataloged functions. A single coding agent session is created using the Claude Agent SDK with access to this directory to generate one function per turn of the session, validating each function against the local stubs before moving on to the next. Once each function has been implemented, the user is shown a display at gate 3 where they can scroll through the code and the workflow file before approval.

**Execution and Gate 4**: At gate 4, the system requests approval to deploy the workflow, and it is deployed using FaaSr to GitHub Actions. Note that, although our prototype targets GitHub Actions, it is trivial to target other platforms using FaaSr (AWS Lambda, OpenWhisk, SLURM, Google Cloud Platform, and Kubernetes).

**Post-run agent and Gate 5**: Finally, if the deployment is successful, the post-run WDA agent serves as the final HITL gate. It provides the user with a natural-language summary of the workflow run and its artifacts. It exposes a chat interface that allows the user to ask interactive questions about the run, revise the workflow, commit their functions to the catalog for future reuse, and save the workflow for later deployment or adaptation. Upon execution failure, the WDA reviews the workflow artifacts and logs and advises the user on a recommended adaptation to submit for revision.

### B. Revision

Upon revision, the system uses an LLM to classify whether the requested changes are structural or functional. Structural changes modify the underlying DAG, whereas functional changes modify functions already in the skeleton. Note that structural changes can only be made at skeleton creation, so structural revisions loop back to the skeleton step, while functional changes loop back to the resolution step.

### C. Function Generation

For function generation, we use the Claude Agent SDK with Opus 4.8. When the MGA is deployed, the WCA provides the skeleton DAG with one of four labels for each function: `[New]`, `[Catalog]`, `[Cache]`, or `[User Provided Module]`; an optional `[Adapt]` label tells the MGA to use an existing function as context but adapt it to the user's requirements. Functions labeled `[Cache]` were implemented

on an earlier revision cycle. Nodes labeled `[Catalog]` or `[Cache]` are skipped by the MGA unless they carry an `[Adapt]` label. For `[User Provided Module]` without `[Adapt]`, the MGA wraps the existing module with FaaSr I/O stubs for persistent data retrieval/storage from/to S3 cloud bucket(s) without changing the module's code, constructs synthetic data, and performs local parity checks of the wrapped function against the original. For `[New]` or `[Adapt]` functions, the MGA generates new code, in the latter case with the original function as context. The MGA is instructed not to return an implementation until it has tested the function using real data where available (produced by upstream functions), publicly available data (via an API), or local synthetic data when neither is available. Local testing is enabled by FaaSr stubs that mirror the actual FaaSr API. The FaaSr workflow JSON itself is created deterministically by means of a tool (not an agent) that translates the skeleton to the schema, and any required dependencies the MGA discovers beyond the WCA's list are written to a manifest and merged into the workflow configuration.

### D. Catalog Implementation and Search

We implement our catalog on disk, where each entry comprises the following fields: a unique **ID**; **name**; natural-language **description**; **inputs** and **outputs**; PyPI **dependencies**; the names (not values) of **secrets**, if applicable; **rank**; **keywords**; **provenance** (the workflow from which the entry originates); and a **usage count**. To index the catalog, we use BM25 [34] on the catalog entry keywords, using the tokenized natural-language description of each function provided by the WCA's skeleton as our search query. This catalog represents a minimal implementation; future work will seek to increase the scalability, fidelity, and correctness of component reuse and make catalogs compliant with FAIR principles.

## V. Experimental Results

To evaluate our prototype, we derived 4 experiments from SeBS-Flow [35], a serverless workflow benchmark suite, as well as 2 real-world examples automating and scaling a workflow that uses an existing Anaerobic Digestion Model (ADM1) implemented in Python (PyADM1). The ADM1 model is used in practice to size anaerobic digesters and forecast biogas production under varying conditions, though applying it to a specific facility requires extensive manual data reconciliation given the structure, frequency, quality, and accuracy of real-world data. The SeBS-Flow studies show the prototype system's ability to fully generate code and deploy a benchmark suite starting from a natural language prompt, while the PyADM1 studies demonstrate the system's ability to incorporate existing, unmodified model code with generated modules for data preprocessing, concurrent execution, summarization, and visualization.

We first recreate three SeBS-Flow workflows fully end-to-end: machine learning, MapReduce, and video analysis. Prompts used in the experiments are shown in the gray text blocks. For experiments 1-3, bolded text was taken directly from [35], and bulleted entries are the additional context added to ensure the full implementation can be recreated. These initial experiments demonstrate the system's ability to generate small workflows fully end-to-end while remaining faithful to strict user specifications. Experiment 4 demonstrates the system's ability to reuse cataloged modules and append additional MGA-implemented modules. Then, in experiment 5, we provide a description of a workflow that processes "raw" data to conform to PyADM1 inputs and execute the model, where the raw data carries faults representative of wastewater treatment plant: laboratory analytes measured weekly instead of the 15-minute frequency required by the simulation, instrumentation spikes as outliers, and units reported in the American convention (MGD, degrees Fahrenheit, mg/L) rather than metric as required by the model. In experiment 6, we extend experiment 5 to incorporate concurrent execution of a parameter sweep of the model across 20 initial states that vary the solids retention time (SRT). SRT is an operational parameter known to govern both digester sizing and methane production, but is not an explicit model input. For experiments 1-4, we compare our results against a ground truth. In 1-3, the ground truth is the original SeBS-Flow outputs; in 4, we manually calculate the correct word counts from the PDF. In experiments 5 and 6, we rely on domain expert knowledge to verify the behavior of data reconciliation and that model outputs respond to SRT in the expected directions. For brevity, we omit the original prompts from [35] for experiments 2 and 3, as well as the file directory names.

### A. Experiments

**Experiment 1 (Machine Learning)**

**MachineLearning: This workload represents a typical training pipeline: It starts with gen generating a dataset, with the number of samples N and the number of features M as input. Then, we train K different classifiers Ci in parallel. We generate N = 500 samples and M = 1024 features, and train K = 2 classifiers: a Support Vector Machine, and a Random Forest, creating two concurrent functions.**

- For the dataset generation, use make_classification with n_samples=500, n_features=1024, n_redundant=0, n_clusters_per_class=2, weights=[0.9,0.1], flip_y=0.1, and random_state=123
- To preprocess use StandardScaler train_test_split with test_size=0.4, random_state=123
- For Random Forest, set max depth to 5 and the number of estimators to 10; do not do any seeding
- SVM must use SVC with kernel=linear and C at 0.025
- For accuracy, use clf.score

**Revisions**

- **Gate 1**: train_classifier should be split into two concurrent functions

In experiment 1, we generate a workflow that creates a synthetic dataset, preprocesses it, and trains SVM and Random Forest (RF) models concurrently. To ensure enough context to reproduce the original results, we include the dataset generation, preprocessing, and hyperparameter configurations in the prompt. In the initial skeleton, the WCA combined the two training functions into a single ranked function. After revision, train_classifier was split into two functions (SVM and RF). Because both the original and the generated RF

implementations were unseeded, we ran 60 invocations of both the original and generated workflows. The accuracy of SVM was identical, and the generated RF had an average accuracy of $0.850 \pm 0.002$, compared with the original's $0.851 \pm 0.006$.

**Experiment 2 (MapReduce)**

**See section 5 of [35] for the full prompt**

- For the input text generation, use [cat, dog, bird, horse, pig], each distributed evenly and shuffled randomly
- Aside from dataset generation, ensure MapReduce specific functions are fully generalizable to vocabularies of any size and frequency

In experiment 2, we generate a workflow that creates a list of words and runs a MapReduce algorithm split into four steps: Split → Map → Shuffle → Reduce. We provide information on the input data set so that the MGA can recreate the results of our SeBS-Flow baseline. The word counts from this workflow matched the SeBS-Flow ground truth verbatim.

**Experiment 3 (Video Analysis)**

**See section 5 of [35] for the full prompt**

- Model: Faster R-CNN ResNet-50 COCO. Files frozen_inference_graph.pb and faster_rcnn_resnet50_coco_2018_01_28.pbtxt, loaded with OpenCV cv2.dnn
- Three external inputs: frozen_inference_graph.pb, faster_rcnn_resnet50_coco_2018_01_28.pbtxt, and video_small.mp4
- Output: each detection's {label, class, score}, keeping score > 0.5; place in VideoAnalysis/
- Provide the label for the object from the COCO 80 dataset using class_id directly (zero-indexed, no offset; e.g. 0=person)

In experiment 3, we generate a workflow that takes as input a frozen inference graph, configurations for a faster R-CNN model trained on COCO, and a highway video, and classifies the objects in the video. In SeBS-Flow, the results were flattened to include only the outputs of a *single* frame; we fixed this manually. For evaluation, we collapsed the correctness classification to whether the highest-probability object detected matched the ground truth for each frame (a car in all cases). The CURATE workflow correctly identified the cars and did not produce any runtime errors.

**Experiment 4 (PDF + MapReduce + Visualize)**

```
Extract the words from words.pdf, split the word
dataset, run MapReduce, visualize the output
```

**Revisions**

- **Gate 1**: split MapReduce into map, shuffle, reduce
- **Gate 2**: reuse split verbatim and use the original rank for reduce and map

In experiment 4, we reuse a subgraph of the previous MapReduce workflow, and extend it to: 1) use inputs from a PDF file rather than synthetic data, and 2) add a visualization function. The result is 4 reused functions (split, map, shuffle, reduce) and 2 new generated functions (extract_words and visualize_output). For our new input, we use a PDF with 5 words randomly distributed between 100 and 1500 occurrences. Note that we use the same vocabulary size as in the original (5), since the SeBS-Flow description ties the parallelism of the reducers to the vocabulary size, and structural revisions are beyond the scope of this experiment. The expected output matched the actual word counts, and the visualization (a histogram of the 5 words) was well formatted and clear.

**Experiment 5 (PyADM1)**

```
Build a 7-step workflow that cleans raw digester
influent data and runs a PyADM1 simulation on it.
1. convert-units: Reverse field units back to ADM1
units: Q MGD→m3/d, T (F)→T (C) (convert AND rename
the column), divide ALL 22 COD columns by 1000 -- both
solubles (S_⋆) and particulates (X_ch, X_pr, X_li,
X_I, ...). Leave only S_IC, S_IN, S_cation, S_anion
untouched.
2. fill-gaps: Forward-fill missing S_cation and S_anion
values.
3. remove-spikes: Detect and replace isolated sensor
spikes in BOTH Q and T with the local median.
4. interpolate: Treat the 26 weekly-held lab columns as
weekly samples and interpolate to 15-min.
5. validate: Check the cleaned influent and
initial-state files; write validation_report.csv.
6. pyadm1: Run the PyADM1 simulation on the cleaned
influent + initial state; write dynamic_out.csv.
7. visualize: Plot the simulation outputs; write
simulation_plots.png.
```

**Revisions**

- **Gate 2**: the folder should be PyADM1-orig/

In experiment 5, the workflow preprocesses "raw" influent data to conform to PyADM1 inputs, runs the model, and visualizes the results. We generate a synthetic "raw" influent input from the dataset distributed with PyADM1 by introducing changes that capture typical issues with raw data that require cleaning: unit differences, data gaps, and outliers. First, we converted the units of flow (Q, $\mathrm{m}^3/\mathrm{day}$ to MGD), temperature (T, C to F), and 22 concentration features ($\mathrm{kg}/\mathrm{m}^3$ to $\mathrm{mg}/\mathrm{L}$). Then, we applied weekly holds to the 26 laboratory features by freezing the first recorded value of each week. Next, we added gaps of 3–7 days in three places to S_cation and S_anion. Finally, we added implausible spikes to T and Q in four places each, simulating erroneous sensor readings. The CURATE-generated workflow was able to address most of the changes in the 5 preprocessing stages before running PyADM1, except the T spikes. T is hard-coded in PyADM1, so the MGA chose to skip the T spike fixes. Interestingly, the WDA caught this and suggested fixing the T spikes and modifying the PyADM1 model, but we chose to forgo this revision to keep the model unmodified. Note that, although the PyADM1 model is reused, the MGA still needed to wrap it with I/O and check parity with the original, driving up token usage.

**Experiment 6 (PyADM1 with concurrency)**

**Same prompt as experiment 5, except for the following changes in steps 5 and 6:**

```
5. vary-inputs: create 20 inputs derived from the
digester_initial that vary SRT
6. pyadm1 (20x): Run the PyADM1 simulation on the
cleaned influent + initial state; write dynamic_out.csv.
```

**Revisions**

- **Gate 5 (WDA suggestion)**: In the pyadm1 function, make the simulation honor the per-rank SRT from digester_initial.csv - read the SRT value and set the digester feed/flow so that hydraulic (and solids) retention time equals SRT (e.g. set q_ad = V_liq / SRT instead of the hard-coded q_ad = 178.4674), so that the 20 ranked runs produce distinct steady states across the 10-60 day sweep.

TABLE I: Experimental Results

| | CURATE | | | | | Baseline |
|---|---|---|---|---|---|---|
| Exp. | Fn Gen. | Fn Reused | Fn Adapt. | Rev. | Success | Success |
| E1 | 4 | 0 | 0 | G1 | ✓ GT | ✓ 3/3 |
| E2 | 5 | 0 | 0 | – | ✓ GT | ✓ 1/3 |
| E3 | 3 | 0 | 0 | – | ✓ GT | ✗ 0/3 |
| E4 | 2 | 4 | 0 | G1, G2 | ✓ GT | ✓ 1/3 |
| E5 | 6 | 1 | 0 | G2 | ✓ EV | ✓ 1/3 |
| E6 | 1 | 4 | 2 | G5 | ✓ EV | ✓ 3/3 |

**Fn Gen**: number of functions generated by the MGA; **Fn Reused**: number of functions reused from the catalog; **Fn Adapt**: number of existing functions adapted to the prompt; **Rev**: gates where revision occurred; **Success**: success of the experiment (free from runtime errors and correct outputs) – for CURATE, GT: ground truth, EV: expert verified; for the baseline, the number of correct (and error-free) runs out of three compared to the verified CURATE outputs

In experiment 6, we scaled the PyADM1 workflow to 20 concurrent invocations to perform a parameter sweep of SRT, which is controlled by flow into the digester and digester volume. We introduced a request in the prompt to vary inputs, modifying the preprocessed influent to use different SRT values. Upon deploying the workflow and reviewing outputs, the WDA identified an issue: q_ad (the feed flow) was a hard-coded value in PyADM1, meaning that the SRT would not change despite the varied input. The WDA made a revision suggestion to PyADM1, which the HITL approved; after the revision, the workflow produced the correct results.

**Baseline:** We compare our approach to a baseline that only uses the coding agent that implements the MGA: a plain Opus 4.8 Claude Code instance spawned using the Claude Agent SDK with the following context:

**Baseline Context**

- http://faasr.io
- Web search
- An example workflow JSON
- A copy of the FaaSr workflow schema
- An explanation of where to place external dependencies in the JSON
- A list of available FaaSr API calls
- The original experiment prompt
- PyADM1 (for E5-6)
- "Create a FaaSr workflow based on the user prompt. Your final output should be a FaaSr JSON file and functions written in Python"

To capture response variance with no room for HITL revision, we ran each baseline experiment 3 times. If any of the 3 runs succeeds (no runtime errors and correct outputs), the experiment is a success. Failures can stem from runtime errors or incorrect output. Every run uses the same context, except for E5-6, where PyADM1 and the raw inputs are provided. In many cases, there were runtime errors related to FaaSr that were not knowable from the provided context alone. For example, in some cases, the baseline agent mapped action names to dependencies instead of function names, or tried to split the action's code into multiple files. One of the FaaSr S3 API operations had an incorrectly documented argument name. In all such cases, we manually corrected the issue. We compare the results of our baseline to the ground truth or expert-verified CURATE outputs.

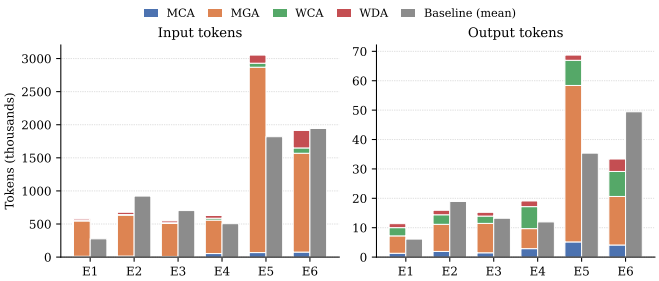



Fig. 3: Token usage per experiment (E1–E6). For each experiment, the left (colored, stacked) bar is the sum over CURATE agents (MCA, MGA, WCA, WDA), and the right (grey) bar is the Claude Code baseline.

## VI. Discussion

Our CURATE prototype demonstrates the *feasibility* of a multi-agent workflow generation system, incorporating existing module reuse and HITL feedback revisions. Compared to a single-agent baseline, token usage (Figure 3) is higher in some cases; however, the baseline showed significant non-determinism in output correctness and, in many cases, did not produce a correct run and required manual revisions. In contrast with CURATE, no such revisions were required aside from the natural language HITL feedback built into the system. CURATE enables module reuse, which can improve the reliability of generated workflows by reusing verified modules. While the potential for decreasing token costs from reuse has been observed in some cases, further experiments are warranted - e.g. between E5 and E6, token usage nearly halved despite increased workflow complexity, while also requiring less HITL oversight of the preprocessing pipeline previously verified in E5. In E6, the post-run WDA successfully caught an issue with the PyADM1 model (where a parameter used to vary SRT was hard-coded), suggested a revision for HITL approval, and routed it back to the WCA/MCA for module resolution, highlighting a benefit of the multi-agent integration. Table I shows the results of each of the experiments, including the baseline results after corrections were applied to errors that were not reasonably knowable from the provided context. Finally, unlike the baseline, our prototype automates the deployment of workflows to a reproducible WMS.

While initial results are promising, our results have limitations. First, the Claude Code baseline did not include HITL revision at each step of the workflow-building process. We chose this comparison because Claude Code does not have a built-in notion of workflow creation, and it does not follow the same DAG sketch → reify → test model. Additionally, SeBS-Flow was originally designed as a performance benchmark rather than a scientific workflow suite; nonetheless, it provided a valuable reference for validating generated workflows. The ADM1 workflow tested is limited to initial data preparation. The full end-to-end workflow entails many additional steps, including preprocessing to achieve mass balance, extensive feature engineering, sensitivity analysis, and validation under unique conditions. However, operationalizing these steps is

expected to be reasonably achievable with CURATE, assuming practicing engineers can explicitly define the assumptions that govern certain actions. This could save a significant number of engineer-hours for mechanistic model calibration and significantly improve model accessibility, reproducibility, and utilization.

## VII. Conclusion

We presented CURATE, a human-in-the-loop multi-agent system that manages workflows across their entire lifecycle, from composition to deployment and broader reuse. We demonstrated the system's feasibility with an initial open-source [32] Python prototype and 6 evaluation experiments. Our prototype faithfully recreated 3 workflows in the SeBS-Flow serverless benchmark suite, reused a MapReduce pipeline from the catalog to create a new workflow, and automated and scaled a non-trivial anaerobic digestion simulation pipeline with preprocessing and visualization. Directions for future work include extending the catalog design to support semantic match queries and mechanisms for sharing and reuse across workflow repositories; interactive user interfaces for visual and conversational workflow generation; extensive user studies across different disciplines; and policies and evaluations focused on resource isolation, such as managing erroneous I/O operations generated by the MGA.

## Acknowledgments

This work was supported by the U.S. National Science Foundation (NSF) under awards OAC-2450241, OAC-2311124, CNS-2533773, and CNS-2310422. Any opinions, findings, and conclusions expressed herein are those of the authors and do not necessarily reflect the views of the NSF. The authors also thank Dr. Nathan Tallent of Pacific Northwest National Laboratory (PNNL) for his feedback.

## References

[1] B. Ludäscher *et al.*, "Scientific workflow management and the kepler system," *Concurr. Comput.: Pract. Exper.*, vol. 18, no. 10, p. 1039–1065, Aug. 2006.

[2] E. Deelman, D. Gannon, M. Shields, and I. Taylor, "Workflows and e-science: An overview of workflow system features and capabilities," *Future Gener. Comput. Syst.*, vol. 25, no. 5, p. 528–540, May 2009.

[3] R. F. da Silva *et al.*, "A community roadmap for scientific workflows research and development," in *Proc. IEEE Workshop on Workflows in Support of Large-Scale Science (WORKS)*, Nov. 2021, p. 81–90.

[4] M. Nejjar, L. Zacharias, F. Stiehle, and I. Weber, "Llms for science: Usage for code generation and data analysis," *Journal of Software: Evolution and Process*, vol. 37, no. 1, p. e2723, 2025.

[5] A. Dhruv and A. Dubey, "Leveraging large language models for code translation and software development in scientific computing," in *Proc. Platform for Advanced Scientific Computing Conf. (PASC)*, 2025, p. 1–9.

[6] J. Xu, W. Du, X. Liu, and X. Li, "Llm4workflow: An llm-based automated workflow model generation tool," in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024, p. 2394–2398.

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[8] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. NeurIPS*, 2024.

[9] S. Yao *et al.*, "React: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.

[11] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," 2025. [Online]. Available: https://arxiv.org/abs/2411.12924

[12] Anthropic, "Claude code," 2025, [Online]. Available: https://www.anthropic.com/claude-code. [Accessed: Jul. 2026].

[13] OpenAI, "Codex," 2025, [Online]. Available: https://openai.com/codex/. [Accessed: Jul. 2026].

[14] O. J. R. Gustafsson *et al.*, "Workflowhub: a registry for computational workflows," *Scientific Data*, vol. 12, no. 1, May 2025.

[15] M. R. Crusoe *et al.*, "Methods included: standardizing computational reuse and portability with the common workflow language," *Communications of the ACM*, vol. 65, no. 6, 2022.

[16] S. R. Wilkinson *et al.*, "Applying the fair principles to computational workflows," *Scientific Data*, vol. 12, no. 1, Feb. 2025.

[17] F. Suter *et al.*, "A terminology for scientific workflow systems," *Future Generation Computer Systems*, vol. 174, p. 107974, 2026.

[18] R. Xu and Y. Yan, "Agent skills for large language models: Architecture, acquisition, security, and the path forward," 2026. [Online]. Available: https://arxiv.org/abs/2602.12430

[19] K. Thareja, H. Safri, R. Mayani, A. Mandal, and E. Deelman, "From specification to execution: Ai assisted scientific workflow management," 2026. [Online]. Available: https://arxiv.org/abs/2606.18425

[20] E. Deelman *et al.*, "Pegasus, a workflow management system for science automation," *Future Gener. Comput. Syst.*, vol. 46, no. C, p. 17–35, May 2015.

[21] C.-T. Lee *et al.*, "Can an llm find its way around a spreadsheet?" in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2025, p. 294–306.

[22] J. Starlinger, B. Brancotte, S. Cohen-Boulakia, and U. Leser, "Similarity search for scientific workflows," *Proc. VLDB Endow.*, vol. 7, no. 12, p. 1143–1154, Aug. 2014.

[23] J. Gottweis *et al.*, "Accelerating scientific discovery with co-scientist," *Nature*, vol. 655, no. 8122, p. 487–496, May 2026.

[24] Anthropic, "Claude science: An AI workbench for scientists," https://www.anthropic.com/news/claude-science, Jun. 2026, accessed: 2026-07-17.

[25] H. Ma, A. Brace, C. Siebenschuh, I. Foster, and A. Ramanathan, "Langchain-parsl: Connect large language model agents to high performance computing resource," in *Proc. SC '25 Workshops*, 2025, p. 78–85.

[26] Y. Babuji *et al.*, "Parsl: Pervasive parallel programming in python," in *Proc. Int. Symp. on High-Performance Parallel and Distributed Computing (HPDC)*, 2019, p. 25–36.

[27] O. Yildiz and T. Peterka, "Do large language models speak scientific workflows?" in *Proc. SC '25 Workshops*, 2025, p. 2225–2233.

[28] W. Shin *et al.*, "The (r)evolution of scientific workflows in the agentic ai era: Towards autonomous science," in *Proc. SC '25 Workshops*, 2025, p. 2305–2316.

[29] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for llm agents," in *Proc. NeurIPS*, 2025.

[30] Model Context Protocol Contributors, "Model context protocol specification," https://modelcontextprotocol.io/specification/2025-11-25, 2025, revision 2025-11-25. Accessed: 2026-07-18.

[31] A. Kamatar *et al.*, "Empowering scientific workflows with federated agents," in *Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS)*, 2026.

[32] N. Cutler, "CURATE-prototype," https://github.com/nolcut/CURATE-prototype, 2026, accessed: Aug. 3, 2026.

[33] S. Park *et al.*, "Faasr: Cross-platform function-as-a-service serverless scientific workflows in r," in *Proc. IEEE Int. Conf. on e-Science*, 2024.

[34] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, p. 333–389, Apr. 2009.

[35] L. Schmid, M. Copik, A. Calotoiu, L. Brandner, A. Koziolek, and T. Hoefler, "Sebs-flow: Benchmarking serverless cloud function workflows," in *Proc. European Conf. on Computer Systems (EuroSys)*, 2025, p. 902–920.